

GSQLi: A GAN-based Approach for Adversarial SQL Injection Sample Generation against WAF

1st Le Minh Khan

University of Information Technology
Vietnam National University
Ho Chi Minh City, Vietnam
20521430@gm.uit.edu.vn

2nd Hien Do Hoang

University of Information Technology
Vietnam National University
Ho Chi Minh City, Vietnam
hiendh@uit.edu.vn

3rd Khoa Ngo-Khanh

University of Information Technology
Vietnam National University
Ho Chi Minh City, Vietnam
khoank@uit.edu.vn

4th Phan The Duy

University of Information Technology
Vietnam National University
Ho Chi Minh City, Vietnam
duypt@uit.edu.vn

5th Van-Hau Pham

University of Information Technology
Vietnam National University
Ho Chi Minh City, Vietnam
haupv@uit.edu.vn
*Corresponding author

Abstract—Web application firewalls (WAFs) are critical for detecting and blocking malicious activities, offering essential protection for web applications. However, to defend against the complexity of modern attacks, penetration testers must regularly evaluate WAFs to identify potential weaknesses. A key aspect of this process involves bypassing WAFs or attack detectors, and utilizing machine learning (ML) significantly enhances the effectiveness of testing. Using ML not only increases the efficiency of identifying vulnerabilities but also reduces the need for manual intervention. In this paper, we propose a novel method named GSQLi to mutate payload for SQL Injection (SQLi) attacks, one of the most popular attacks on web applications, to deceive WAFs or detectors. By leveraging the Generative Adversarial Network (GAN), SQLi payloads are generated by applying several mutations on the original ones, making it challenging for detectors to identify them as malicious. Additionally, these mutated payloads retain their ability to exploit vulnerabilities effectively. Experimental results prove the capability of payloads generated by our approach to bypassing machine learning-based attack detectors and ModSecurity, a real-world WAF. This reveals potential vulnerabilities in current defense systems, enabling defenders to address weaknesses more swiftly and enhance protection against advanced attacks on web applications.

Index Terms—GAN, SQL Injection, Adversarial Sample Generation, Security Pen-Testing

I. INTRODUCTION

Recently, the rise in the quantity of web applications as well as their managed valuable data has led to increased security risk and attacks targeting this type of application [1] [2].

Wherein, SQL injection (SQLi) is a highly prevalent web vulnerability that can manipulate database queries by injecting malicious code into vulnerable web applications, leading to data breaches, unauthorized access, and severe financial and reputational damage. More surprisingly, the time to detect a vulnerability and launch an SQLi attack averages less than 10 seconds. Once an SQLi vulnerability is successfully exploited,

attackers can obtain all stored data, manipulate or destroy it, and even worse, gain full control of the SQL server. Due to its high risk and widespread occurrence, SQLi is consistently ranked among the TOP 10 web vulnerabilities as published by OWASP [3].

To defend against web attacks, WAFs and attack filters are commonly used to protect against malicious attacks. However, they remain vulnerable to many of today's highly sophisticated attack techniques. As a result, penetration testing of these WAFs and filters is essential to ensure that they can effectively detect and block such advanced threats.

Numerous studies have utilized machine learning (ML) to enhance the efficiency and automation of penetration testing [4] [5] [6]. Current models often use reinforcement learning (RL) or Monte Carlo Tree Search (MCTS) to develop optimal mutation strategies for attack payloads, primarily focusing on generating effective mutations. Another approach involves adversarial learning models to mutate SQLi samples; however, these mutations often lack strategic planning and fail to guarantee the effectiveness or integrity of the resulting attacks.

Motivated by the above issues, this paper introduces a novel method that leverages Generative Adversarial Networks (GANs) to mutate SQL injection (SQLi) payloads in ways that effectively evade detection by defense systems. By using GANs, the approach automates the generation of highly diverse and obfuscated attack payloads, allowing penetration testers to reduce the manual effort required to craft new attack vectors. Furthermore, the integration of machine learning significantly enhances the method's ability to bypass modern security systems, increasing the likelihood of uncovering previously unknown vulnerabilities. This automated, intelligent approach not only streamlines the penetration testing process but also strengthens the ability to identify new weaknesses

in web applications, making it a powerful tool for enhancing cybersecurity defenses.

The rest of this paper is constructed as follows. **Section II** reviews some recent related work in the field of evaluating WAF, especially using ML/DL methods. In **Section III**, our proposed GSQLi is described in detail, followed by designed experiments and assessment results mentioned in **Section IV**. Finally, **Section V** concludes the paper.

II. RELATED WORK

There has been a lot of research on methods to assess web defense systems. One notable research by Yuting Guan et al. [5] leveraged RL techniques to mutate SQLi payloads, using the Soft Actor-Critic algorithm (SAC)-based actor and static and dynamic vectors for the preprocessing of original payloads. Particularly, upon reverting to the original format, attack payloads were modified by several mutating strategies such as adding characters or replacing them with targets to evade WAFs and retain the functionality and goal of the original attack. Besides, research [4] focused on generating XSS attack payloads utilizing the GAN model. That work segmented parts of an XSS attack payload and classified them according to their corresponding positions, such as the head, body, and tail of the payloads, to create a training dataset. New payloads were then created by combining these segments to form complete samples with the Monte Carlo (MC) search technique. Moreover, Zhenqing Qu et al. [6] conducted a study related to SQLi attack payload generation using logic transformation rules to alter the character sequences of SQL query strings to modify the attack string format. Specifically, they analyzed SQLi attack strings into tokens and classified them according to their roles in the query. The authors then applied rules known as Context-free Grammar (CFG) to these tokens and employed the MCTS method to identify optimal replacements to evade detection.

Though the above works were able to create attack samples designed to deceive and bypass attack detection systems while maintaining the attack's effectiveness, there is room for investigating the potential of applying GAN in creating SQLi payloads. In addition, there is also a lack of evaluation on the

capability of real-world attacks using crafted payloads, which are the motivations of our work.

III. GSQLi - PROPOSED METHOD

A. Overall Architecture

GSQLi leverages GAN to generate SQLi payloads that can deceive attack detectors, with the architecture depicted in **Fig. 1**. The generator generates new attack payloads by changing mutation actions and applying them to the original ones to avoid detection. Besides, the discriminator attempts to evaluate whether the generated payloads are normal (undetected attacks) or malicious activities. Through iterative learning, the generator improves its ability to mutate payloads that closely resemble undetected attacks while evading the defenses.

Initially, a payload is put into Token Parser, resulting in corresponding Tokens and a Mutation Vector. Next, the Mutation Vector is combined with noise to become an input of the Generator (G) to generate appropriate Mutation Actions. These actions are applied to the original payload by the Payload Transformer to produce a Mutated Payload, which is then fed into the Classifier for classification. Subsequently, the Discriminator (D) is trained with input including the Mutation Actions, Mutation Vector, and the label obtained from the Classifier. This process is repeated until the Discriminator can not detect the attack payloads mutated by the Generator.

B. Payload Preprocessing

SQLi payload is split into smaller parts called *tokens*. These tokens are then categorized into characters, numbers, keywords, operations, expressions, strings, comments, functions, and bare words (names of tables or columns, etc.). This token is crucial for reassembling new payload in the transformation process.

After processing tokens, we create a *Mutation Vector* by counting values from tokens followed by predefined features shown in **Table I**.

C. Payload Mutation with Generative Adversarial Network

Our GSQLi framework generates SQL injection (SQLi) payloads capable of evading web application firewalls (WAFs)

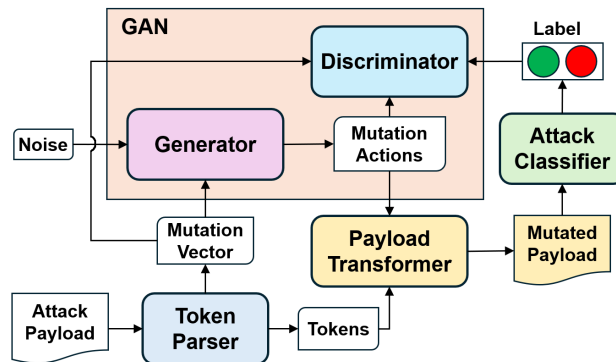


Fig. 1. GSQLi Architecture.

TABLE I
FEATURES OF MUTATION VECTOR

No.	Feature	Description
1	Union	The number of UNION keywords
2	Where	The number of WHERE keywords
3	Unlike	The number of UNLIKE keywords
4	Like	The number of LIKE keywords
5	Space	The number of spaces
6	TrueExp	The number of true expressions
7	FalseExp	The number of false expressions
8	TrueVal	The number of true values
9	FalseVal	The number of false values
10	And	The number of AND operations
11	Or	The number of OR operations
12	Asterisk	With (1) or without (0) any asterisk
13	Comment	The number of comments
14	NoFunc	The number of functions
15	NoBareword	The number of bare words

while maintaining their original functionality. Each payload possesses unique characteristics, represented by its own vector. Instead of incorporating only random noise, we utilize a mutation vector as the additional input to the GAN model. This approach allows the GAN to generate tailored mutation strategies for each specific payload. To accomplish this, we customized a GAN model based on the conditional GAN architecture [7], adapting it specifically for our objectives of crafting evasive SQLi attacks.

In more detail, the Generator in our model takes both a noise vector and a mutation vector as inputs to generate specific mutation actions. These actions serve a dual purpose, firstly they are used to train the Discriminator and secondly to mutate tokens to create new payloads. The Discriminator, in turn, is trained using the mutation vector and the generated mutation actions as inputs, with its output being a classification label indicating whether the mutated payload is adversarial or not.

During the training process, the Generator aims to produce samples that are classified as normal. Meanwhile, the Discriminator is trained to mimic the behavior of the classifier, distinguishing between normal (labeled as 0) and malicious (labeled as 1) samples to improve its ability to detect adversarial payloads. Using this approach, the loss functions for both the Generator (L_G) and the Discriminator (L_D) are calculated according to **Eq. (1)** and **Eq. (2)**, respectively. Note that, all generated samples are malicious and we try to make them the same as normal ones.

$$L_G = CE(y_0, D(v, a)) \quad (1)$$

$$L_D = CE(y_c, D(v, a)) \quad (2)$$

where CE is the cross-entropy function, v , and a present for Mutation Vector, and Mutation Actions, respectively. y_0 is normal prediction and y_c is the prediction of the classifier.

Mutation Actions, which are the output of the Generator, present the actions of transforming tokens to generate new payloads. We used mutation actions defined in [6] and some customized ones that are summarised in **Table II**. Original samples applying these mutated strategies can also preserve

their functionality and logic [6]. Each token is associated with several possible mutation actions, which the Generator selects from during the mutation process. Since a payload consists of multiple tokens, the Generator produces a corresponding set of actions for each token, leading to a variety of mutation outcomes.

D. Payload Transformer

This component is responsible for transferring original payloads to new payloads based on the mutated actions generated by the Generator. Particularly, the extracted tokens from the Token Parser have corresponding mutation actions, then the Payload Transformer generates the new payload following these instructions.

E. Attack Classifier

The Attack Classifier is a component to label the mutated samples for the Discriminator. Note that, all generated samples are SQLi attack samples. However, using a classifier that has the knowledge to identify normal and malicious payloads can help train GAN models more efficiently. Mutated samples that can deceive the classifier can also deceive the Discriminator. Therefore, the GAN model converges better because the loss value is optimal.

IV. EXPERIMENTS AND EVALUATION

A. Environmental setup

Our experiments are deployed on a Ubuntu v22.04 machine with 6 CPU cores, 32GB RAM, installed Python v3.10, and Tensorflow v2.16.1. To implement the Token Parser, we utilize the Libinjection [8]. This library parses SQLi payloads into tokens, which are then used to generate mutation vectors and create inputs for the Payload Transformer. Besides, the GAN model is built with a Generator and a Discriminator using DL, consisting of multiple neural layers shown in **Table III**. Notations n , v , and a present for the size of Noise, Mutation Vector, and Mutation Actions, respectively. Additionally, the Attack Classifier is constructed using a CNN model. The payload is first transformed into a sequence of integers, which are then passed through an embedding layer to generate a vector representation. This resulting embedding vector serves as the input to the CNN model.

B. Datasets

We utilize two datasets for our experiments: the HttpParams dataset [9] and the SSHS dataset [10] from the Kaggle repository. Since our paper focuses on SQL injection (SQLi) attacks, we use normal and SQLi payloads. In both datasets, we selected SQLi payloads shorter than 100 characters due to limitations in the Libinjection library. The distribution of these payloads is presented in **Table IV**. Additionally, we use the HttpParam dataset to train the GAN model and both datasets for evaluation.

Additionally, payloads from the HttpParams dataset with lengths exceeding 100 characters are utilized to train the classifier model. To process long payloads, we can split them

TABLE II
MUTATION METHODS AND EXAMPLES

Mutation Method	Example
Case Swapping	UNION → uNIoN
Inline Comment	UNION SELECT → /*!UNION*/ /*!SELECT*/ , UNION → /*!50000UNION*/
Where Injection	WHERE id=2 → WHERE TRUE and id=2 , WHERE id=2 → WHERE 1<>1 or id=2
Whitespace Swapping	OR 1 = 1 → OR/*123abc*/1/*123abc*/=/*123abc*/1
Logical Operators Swapping	and TRUE → && TRUE , OR 1=2 → 1=2
Compare Operators Swapping	1=2 → 1 LIKE 2 , 1!=2 → 1 UN LIKE 2 , 1!=1 → 1 <> 2
Number Encoding	10=1 → 0xA=0x1
String Encoding	WHERE username = "admin" → WHERE username = 0x61646D696E
Logic Constant	TRUE → 1=1 , TRUE → 1!=2 OR 2!=2 OR 1=3 , FALSE → 1!=2 AND 2!=2 AND 1=3

TABLE III
LAYERS OF GENERATOR AND DISCRIMINATOR

Generator		Discriminator	
Layer	Output Shape	Layer	Output Shape
Input	(None, n)	Input	(None, v)
Input	(None, v)	Input	(None, a)
Concatenate	(None, $n + v$)	Concatenate	(None, $v + a$)
Dense	(None, 512)	Dense	(None, 256)
Normalization	(None, 512)	Normalization	(None, 256)
Dense	(None, 256)	Dense	(None, 128)
Normalization	(None, 256)	Normalization	(None, 128)
Dense	(None, 128)	Dense	(None, 64)
Normalization	(None, 128)	Normalization	(None, 64)
Dense	(None, a)	Dense	(None, 2)

TABLE IV
THE SAMPLE DISTRIBUTION OF THE TWO DATASETS

Dataset	No. Normal samples	No. SQLi samples
HttpParams	19,304	5,557
SSHS	19,573	6,217

into segments for tokenization, and then tokened segments are reassembled for mutation. This model is then employed to classify the mutated SQLi payloads effectively.

C. Evaluation metrics and scenarios

1) *Metrics*: We use TPR (True Positive Rate) and FNR (False Negative Rate) metrics to evaluate the deception ability of GSQLi on detectors. TPR (Eq. (3)) is the ratio of accurately detected malicious payloads to the total malicious ones. FNR (Eq. (4)) presents the ratio of undetected malicious payloads to the total malicious ones.

$$TPR = \frac{TP}{TP + FN} \quad (3)$$

$$FNR = \frac{FN}{FN + TP} \quad (4)$$

2) *Scenarios*: To evaluate the performance of GSQLi, we trained the GAN model using the HttpParams dataset and conducted experiments across three different scenarios.

- 1) **Scenario 1**: The objective of this scenario was to test GSQLi's capacity to produce a large number of unique mutated payloads from the original dataset. Starting with

5,557 original samples from the HttpParams dataset, we introduced random noise to generate a diverse set of mutant payloads, assessing the model's ability to create distinct variations from the initial inputs.

- 2) **Scenario 2**: In this scenario, we generated mutated samples from both datasets and evaluated them against three DL-based classifiers: RNN, GRU, and BiLSTM. The performance was measured using TPR and FNR metrics. The goal was to assess the ability of the mutated payloads to deceive these ML-based classifiers and to demonstrate whether the model can generalize to unseen payloads not present in the training set. Each test was repeated five times for every input, and the average results were taken to ensure the stability and reliability of the model's performance.
- 3) **Scenario 3**: In the third scenario, we use the mutated payloads generated in Scenario 2 and tested them against ModSecurity installed the OWASP rule set. The goal was to evaluate the evasion capabilities of these mutated payloads in bypassing real-world WAFs.

D. Evaluation results

1) *Scenario 1*: GSQLi is capable of generating a significantly larger volume of new samples compared to the original dataset. A comparison between the original samples, input noise, and the newly generated payloads is illustrated in Fig. 2. Rather than focusing on a single optimal action for each payload, GSQLi generates a combination of optimal actions. This approach allows multiple Mutation Actions to be applied to a single original payload, resulting in a variety of mutated payloads.

2) *Scenario 2*: The results in the Table V highlight the effectiveness of our method in generating mutated payloads that can deceive ML-based classifiers. While the original samples show near-perfect detection rates across all models, the mutated samples reveal a noticeable decline in True Positive Rates (TPR) and an increase in False Negative Rates (FNR), indicating that the mutated payloads are more challenging for the classifiers to detect. This is especially evident on the SSHS dataset, where the TPR drops significantly to 87.5% with RNN, 88.9% with GRU, and 88.3% with BiLSTM, with corresponding FNRs increasing to 12.5%, 11.1%, and 11.7%,

TABLE V
TPR AND FNR (%) OF ML-BASED CLASSIFIERS ON ORIGINAL AND MUTATED PAYLOADS

Dataset	Original Samples				Mutated Samples			
	Http Params		SSHS		Http Params		SSHS	
Metrics	TPR	FNR	TPR	FNR	TPR	FNR	TPR	FNR
RNN	99.95	0.05	97.54	2.46	99.4	0.6	87.5	12.5
GRU	99.95	0.05	99.87	0.13	99.83	0.17	88.9	11.1
BiLSTM	99.95	0.05	99.69	0.31	99.87	0.13	88.3	11.7

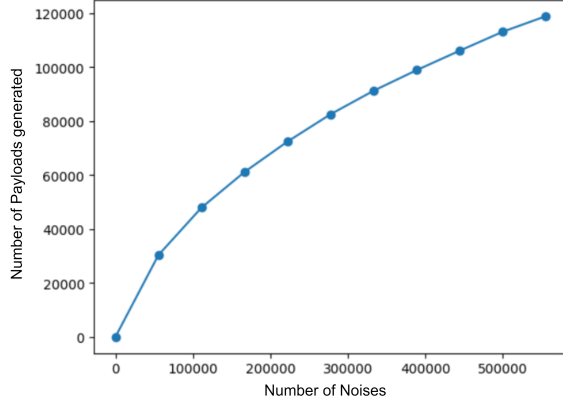


Fig. 2. The number of unique mutated payloads generated from random noises.

respectively. These findings demonstrate that our method successfully generates new patterns that effectively evade detection by ML-based firewalls, particularly excelling in fooling classifiers on more complex datasets like SSHS.

3) *Scenario 3*: The experimental results, shown in **Table VI**, demonstrate the evasion capabilities of GSQLi-generated mutated payloads when tested against ModSecurity, a widely used WAF. The FNR values, 0.19% for the HttpParams dataset and 2.3% for the SSHS dataset indicate that a significant number of mutated payloads can bypass ModSecurity without detection. These results emphasize the effectiveness of this method in fooling real-world WAFs, showing that the mutated payloads successfully evade detection while maintaining attack functionality. This highlights GSQLi's potential as a valuable tool for pen-testers to identify weaknesses in WAFs.

E. Comparison with other related works

Our GSQLi focuses on generating SQL injection (SQLi) attack payloads using a customized GAN, setting it apart from other approaches in both scope and effectiveness. As can be seen in **Table VII**, while Zhenqing and Yuting also target SQLi, they rely on CFG, MCTS, and SAC, which offer different mechanisms for attack generation. In comparison to Zhang and Ankur, who explore Cross-Site Scripting (XSS)

attacks, our method specifically addresses SQLi, providing more targeted results. Unlike Chowdhary, whose approach using GAN lacks payload functionality checks, our model ensures that generated payloads are functional. Moreover, our work, like Qu and Guan's, is tested in both ML-based and real-world WAF environments, making it more comprehensive and practical for real-world applications, while also maintaining functionality checks to ensure high-quality attacks.

V. CONCLUSION

This paper introduces a novel approach for generating SQLi attack payloads that can produce a large variety of new, evasive payloads while retaining their ability to exploit vulnerabilities. The proposed method uses GAN and pre-defined mutated actions to craft these payloads, enabling pen-testers to evade detection systems such as WAFs and ML-based detectors. Experimental testing of the generated SQLi payloads was conducted, demonstrating the ability of the proposed approach to bypass both ML-based classifiers and real-world WAFs, such as ModSecurity. The results not only highlight the effectiveness of this attack generation method but also underscore its potential applications in penetration testing and security evaluation. In future work, we extend the model to generate adversarial payloads for other types of web-based attacks, such as Cross-Site Scripting (XSS) and Remote Code Execution (RCE), etc. Additionally, integrating reinforcement learning with GANs could further optimize the payload generation process by adapting to real-time defenses.

ACKNOWLEDGMENT

This research is funded by University of Information Technology - Vietnam National University HoChiMinh City under grant number of D1-2024-60.

REFERENCES

- [1] Radware, "2024 Global Threat Analysis Report Executive Summary: Analysis of the most significant cybersecurity events and trends of 2023," Tech. Rep., 2024.
- [2] M. Tremante, S. Zejnilovic, and C. Newcomb, "Application Security report: 2024 update," CloudFlare, Tech. Rep., 2024.
- [3] OWASP, "OWASP Top Ten," <https://owasp.org/www-project-top-ten/>, 2024.
- [4] A. Chowdhary, K. Jha, and M. Zhao, "Generative Adversarial Network (GAN)-Based Autonomous Penetration Testing for Web Applications," *Sensors*, vol. 23, no. 18, p. 8014, 2023.
- [5] Y. Guan, J. He, T. Li, H. Zhao, and B. Ma, "SSQLi: A Black-Box Adversarial Attack Method for SQL Injection Based on Reinforcement Learning," *Future Internet*, vol. 15, no. 4, p. 133, 2023.
- [6] Z. Qu, X. Ling, T. Wang, X. Chen, S. Ji, and C. Wu, "AdvSQLi: Generating Adversarial SQL Injections against Real-world WAF-as-a-service," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 2623 – 2638, 2024.

TABLE VI
FNR (%) OF MODSECURITY ON THE MUTATED PAYLOADS

Dataset	HttpParams	SSHS
ModSecurity	0.19	2.3

TABLE VII
COMPARISON WITH OTHER WORKS

Work	Attack Type	Model Architecture	Payload Functionality Check	Deception Environment
Zhenqing Qu [6]	SQLi	CFG, MCTS	Yes	ML-based and Real-world WAFs
Yuting Guan [5]	SQLi	SAC (RL)	Yes	ML-based and Real-world WAFs
Zhang [11]	XSS	GAN, MCTS	Yes	ML-based WAFs
Ankur Chowdhary [4]	XSS	Vanilla GAN	No	ML-based and Real-world WAFs
Ours	SQLi	Customized GAN	Yes	ML-based and Real-world WAFs

- [7] M. Mirza and S. Osindero, "Conditional generative adversarial nets," 2014. [Online]. Available: <https://arxiv.org/abs/1411.1784>
- [8] N. Galbreath, "libinjection: The SQL/SQLi detection library," 2012. [Online]. Available: <https://github.com/libinjection/libinjection>
- [9] Morzeux, "HttpParams Dataset," 2017. [Online]. Available: <https://github.com/Morzeux/HttpParamsDataset>
- [10] S. S. H. Shah, "SQL Injection Dataset," 2021. [Online]. Available: <https://www.kaggle.com/syedsaqlainhussain/sql-injection-dataset>
- [11] X. Zhang, Y. Zhou, S. Pei, J. Zhuge, and J. Chen, "Adversarial Examples Detection for XSS Attacks Based on Generative Adversarial Networks," *IEEE Access*, vol. 8, pp. 10 989–10 996, 2020.